SCHUNK Tec-Interview – Practice Questions and Short Answers

Maryam Al Obaidi · Interview: 21.10.2026, 10:00, Microsoft Teams · with Mr. Salas (Software Development)

How to use this file: read each question, answer it out loud first, then compare with the short answer. Use your own words and do not memorize word for word. Replace every [bracket] with a real example from your own work.

1. About me and the job

Q: Tell me about yourself. A: I am a software developer, mostly with C# and .NET, and SQL Server databases. I have a Computer Science degree from the University of Baghdad, and I live in Barcelona. [Add one sentence about your work experience and one project.] I am learning Docker and CI/CD, and I want to work with software that connects to real products.

Q: Why do you want to work in industrial automation? A: I like to see how software controls hardware. In this job my code controls a real product, not only a screen. I love challenges, I learn new technology quickly, and I am open to working with an international team.

Q: What do you know about SCHUNK? A: SCHUNK is a family-owned German company, with its headquarters in Lauffen am Neckar. It is a world leader in gripping technology, clamping technology, and automation. It has about 3,700 employees and works in more than 50 countries. The software team builds cross-platform apps and software for collaborative robots, and works close to the hardware. [Add one thing you read on the website.]

Q: Do you have experience with Flutter or Dart? A: No, I don't have any experience with Dart or Flutter yet. But I have a solid base in C# and object-oriented programming, and I think Dart is very similar to C#. I learn new tools quickly. For example, I learned Docker and CI/CD by myself in a short time. I would start with the official documentation and a small project.

Q: What do you know about communication protocols? A: I know HTTP because I work with REST APIs, and I understand how data is transferred between the browser and the server. I also know the difference between TCP and UDP. I understand the main idea of protocols that transfer data between devices. I don't have experience with industrial protocols like Modbus or CAN yet, but I would like to learn them with your team.

Q: Have you worked with hardware or devices? A: Not professionally yet. This is one of the reasons I want this job. I understand that software must talk to devices through protocols, and I am ready to learn the hardware side with the team.

Q: How would you work with a team in Germany? A: I would communicate clearly and early. I would write short updates, ask questions when I am not sure, and use Git and pull requests so everyone can review the work. I already work with languages and cultures, so I adapt easily.

Q: Do you speak German? A: Not yet. My languages are Arabic, Spanish, English, and basic Catalan. I am open to learning German.

Q: What is a user story? A: A short description of a feature from the user's point of view: "As a user, I want to do something, so that I get a benefit." It helps the team understand what to build and why.

Q: What are your strengths and your weakness? A: My strengths are fast learning, problem solving, and a practical attitude. My weakness is that I am still at a junior to mid level, so I am still learning some advanced topics. To improve, I study every day and I build real projects.

2. Object-oriented programming (OOP)

Q: What is object-oriented programming? A: OOP is a way to write code using objects. An object has data and actions. For example, a Customer object has a name and an email, and it can place an order. OOP keeps code clean, organized, and easy to reuse. The four main ideas are: encapsulation (keep data safe inside the class), inheritance (one class uses the code of another), polymorphism (the same method works in different ways), and abstraction (show only what is important).

Q: What is the difference between an interface and an abstract class? A: An interface is like a contract. It says what a class must do, but not how. An abstract class is a base class that I cannot use directly. It can have ready code and methods that child classes must write. A class can use many interfaces, but it can inherit from only one class. I use an interface for a simple contract, and an abstract class when classes share common code.

3. SOLID principles

Q: What are the SOLID principles? A: SOLID is a set of five rules for clean code.

- **S – Single Responsibility:** one class does one job only.
- **O – Open/Closed:** I can add new features without changing the old code.

- **L – Liskov Substitution:** a child class must work in place of its parent class without problems.
- **I – Interface Segregation:** I prefer small interfaces, not one big interface with many methods.
- **D – Dependency Inversion:** my code depends on interfaces, not on exact classes. This makes testing easier.

Q: Which principle do you use the most? A: Single Responsibility, because it keeps my classes small and clear.

Q: What is dependency injection? A: A class does not create its dependencies by itself. They are given to it, usually through the constructor, and usually as interfaces. This makes the code easier to test and to change. It is connected to the last SOLID principle.

4. C# fundamentals

Q: What is the difference between a value type and a reference type? A: A value type keeps its value directly, like int, bool, and struct. When I copy it, I get an independent copy. A reference type keeps a reference to an object in memory, like a class or an array. When I copy it, both variables point to the same object.

Q: What is the difference between an array and a List? A: An array has a fixed size. A List can grow and shrink, and it has useful methods like Add, Remove, and Contains. I use an array when the size is fixed, and a List when it can change.

Q: What is the difference between string and StringBuilder? A: A string is immutable, so every change creates a new string. StringBuilder changes the same object, so it is faster when I add text many times, for example in a loop.

Q: How do you handle exceptions? A: I use try, catch, and finally. The code that can fail goes in try. In catch I handle the specific error and log it. The finally block always runs, so I use it to close resources. I catch specific exceptions, not only the general Exception.

Q: What is LINQ? A: LINQ is a way to query collections in C# with short and readable code. For example, Where filters, Select transforms, Distinct removes duplicates, OrderBy sorts, and Sum, Max, and Count calculate values. I can solve problems with a loop, and also with LINQ to make the code shorter.

Q: What are async and await? A: They are used for asynchronous code, so the program does not block while it waits for something slow, like a web request or a database call. [Say honestly how much you used it.] I understand the idea and I would like to practice more.

Q: What is a unit test? A: A small automatic test that checks one part of the code, for example one method. It helps me find errors early and change the code safely. [Say honestly if you have written tests.]

5. Git, GitHub, CI/CD and Docker

Q: What is the difference between Git and GitHub? A: Git is a tool on my computer. It saves the history of my code, so I can go back if there is a problem. GitHub is a website where I keep my Git projects online, so my team can see them and work together.

Q: How do you use Git in a team? A: I use a branch for each new task, and I make small commits with clear messages. When I finish, I push my branch and open a pull request. A teammate reviews it, and then we merge it into the main branch. If there is a conflict, I read both changes carefully and keep the correct code. If I am not sure, I ask my teammate.

Q: What is CI/CD? A: CI/CD is a way to build, test, and deliver software automatically. CI means Continuous Integration: every time I push code, the system builds the project and runs the tests, so I find errors early. CD means Continuous Delivery: after the tests pass, the system prepares the new version, and it can publish it automatically. [Say what you really built, for example a simple pipeline with GitHub Actions.]

Q: What is Docker? A: Docker puts an application and everything it needs into a container, so it runs in the same way on any computer. I learned Docker by myself and I want to use it more.

6. Networking

Q: What is the difference between TCP and UDP? A: TCP is reliable. It makes a connection first, checks that all the data arrives in the correct order, and sends again if something is lost. It is slower but safe. Websites, emails, and file transfer use TCP. UDP is fast but not reliable. It sends data without a connection and does not check if it arrived. Video calls and online games use UDP. I choose TCP when every piece of data is important, and UDP when speed is more important.

Q: What is a REST API? A: It is a way for two programs to talk through HTTP. The client sends a request, like GET or POST, to a URL, and the server sends a response, usually in JSON.

7. SQL Server

Q: What is the difference between INNER JOIN and LEFT JOIN? A: INNER JOIN returns only the rows that have a match in both tables. LEFT JOIN returns all the rows from the left table and the matching rows from the right table. If there is no match, the right side is NULL.

Q: What is a primary key and a foreign key? A: A primary key identifies each row in a table in a unique way. A foreign key is a column that points to the primary key of another table, so it creates a relationship between the two tables.

Q: What is an index? A: An index helps the database find rows faster, like the index of a book. It makes reading faster, but it uses extra space and makes writing a little slower.

Q: What is the difference between WHERE and HAVING? A: WHERE filters rows before the grouping. HAVING filters the groups after GROUP BY.

Q: How do you make a slow query faster? A: I check the execution plan, add the right index, select only the columns I need, and avoid unnecessary joins.

8. Debugging and behavioral questions

Q: How do you solve a difficult bug? A: First I repeat the problem, so I know exactly when it happens. Then I read the error message and check the logs. I test small parts of the code one by one, and I can use the debugger. If I still cannot find it, I search online or ask a teammate. After the fix, I test again to be sure nothing else is broken. [Add a real example from your own project.]

Q: Tell me about a mistake you made. What did you learn? A: [Choose a real, small mistake.] I found the problem, I fixed it, and I learned to test more cases before I finish, especially empty values and special cases.

Q: Tell me about a time you learned something new quickly. A: I learned Docker and CI/CD by myself in a short time. I started with the documentation, built a small example, and then used it in my own project.

Q: How do you accept feedback? A: I listen carefully, I ask questions if something is not clear, and I apply the feedback. Feedback helps me improve faster.

Q: Where do you see yourself in three years? A: As a stronger developer in this team, with good knowledge of the products, the hardware side, and the tools, and ready to take more responsibility.

9. Salary question

Q: What are your salary expectations? A: I am open to hearing any offer, and I am ready to discuss it. The role, the team, and the chance to learn are very important for me. I would like to understand the full package and the responsibilities, and then we can discuss it together.

Q: If they insist on a number. A: I prefer to hear your budget for this position first. I am sure we can find a fair solution for both sides.

Do not give any range or number in this interview. If they send an offer, ask for time to read it carefully.

10. Problem solving: the steps and the ideas

Always follow this order:

1. Ask clarifying questions (null or empty input? uppercase and lowercase? duplicates? sorted?).
2. Say your first simple idea out loud, even if it is slow.
3. Write the code.
4. Test it with normal examples and special cases: null, empty, one element, first, last.
5. Say the time, and how to make it faster.

Useful sentences:

- "Could you repeat the question, please?"
- "Let me think for a moment."
- "My first idea is simple, but it is slow for big lists. Can I improve it?"
- "I am not sure, but I think it works like this..."

Short idea for each problem:

- **Reverse a string:** convert it to a character array, reverse it, create a new string. Or loop from the last character to the first.
- **Palindrome:** compare the first character with the last, then move to the center. I only need half of the word.
- **Two Sum:** two loops, with the second loop starting from i + 1. Faster: a dictionary that remembers the numbers I saw, and I look for target minus the current number.
- **FizzBuzz:** check divisible by 3 and 5 first, then 3, then 5, then print the number. The order is important.
- **Count the letters:** a dictionary with the letter as the key and the count as the value.

- **Remove duplicates:** a HashSet. To keep the order, add to the result only when the set accepts the number.
- **Missing number:** N is the count plus one. Expected sum N times N plus 1 divided by 2, minus the real sum. Or a HashSet and check from 1 to N.
- **Valid brackets:** a stack and a dictionary (closing bracket to opening bracket). Push opening brackets. At a closing bracket, if the stack is empty return false, pop and compare. At the end, the stack must be empty.
- **Anagrams:** return false if null, empty, or different length. Convert to lowercase. Count the first word in a dictionary, subtract with the second word, and at the end all counts must be zero. Or sort both words and compare.
- **Second largest number:** return null if the list is null. Remove duplicates, sort, and if there are fewer than two numbers return null, otherwise take the one before the last. Faster: one pass with two variables.
- **First unique character:** pass 1: count each character. Pass 2: go through the original string and return the first character with count 1.

11. Questions to ask them

1. What does a normal day look like in the team?
2. How do the teams in Mataró and Germany work together?
3. Which technologies do you use now, and is Flutter used in this team?
4. How much do developers work with the real hardware?
5. How does the team support new developers in the first months?
6. Is this position hybrid? How many days per week are in the office in Mataró?
7. What are the next steps after this interview?

12. Before the call

- Open the Teams link 10 minutes early and test sound and camera.
- Quiet room, light in front of your face.
- Job description and CV open in front of you, water, paper, and pen.
- Speak slowly. It is fine to ask for a question again. Be honest about what you do not know yet, and show that you learn fast.

Good luck, Maryam! 🍀